Part G: Hardware interfacing and address decoding

G.1 What's this all about? 2
G.2 Learning objectives 3
G.3 Background 5
G.4 Memory-mapped devices 14
G.5 Memory-to-CPU interface 15
G.6 Timing diagrams 23
G.7 Direct memory access (DMA) 29
G.8 Address decoding 32
G.9 Summary 56

G.1 What's this all about?

- How does the microprocessor/microcontroller **interface** with other parts of the computer system? *e.g.*, components that facilitate communication with peripheral devices.
- We are already familiar with the *concept* of the memory system, but now we will look at more detail on the **hardware** interface.
- Many non-memory hardware devices are actually mapped to memory space, which means the CPU accesses them via addresses.

G.2 Learning objectives

By the end of this section of the course, you should be able to:

- ☐ *Define*, in your own words, the terms active-high, active-low, asserted, negated, tri-state.
- ☐ *Explain* what it means for devices to be memory-mapped.
- ☐ *Describe* the relationship between memory port size, data bus size, and *explain* how any particular byte, halfword, or word is accessed.
- ☐ *Read* and *interpret* timing diagrams for various hardware peripherals.
- ☐ *Define* direct memory access; *explain* its advantages and *describe* its basic operation.
- ☐ *Explain*, in your own words, what full and partial address decoding are.
- ☐ *Explain* how partial decoding leads to multiple copies being mapped into memory space; *discuss* advantages and disadvantages.
- ☐ *Explain* the relationship between chip select signals and address decoding; *describe* the general goals for the design of the address decoder.

- ☐ Given specifications for data and address buses and mappings for devices or memory modules, *design* full or partial address decoders; *explain* what mixed-mode decoding is; *design* mixed-mode address decoders.

Resources:

- STM32 microcontrollers programming manual
- STM32 microcontrollers reference manual
- STM32L432KC microcontroller datasheet
- STM32 Nucleo-32 boards user manual

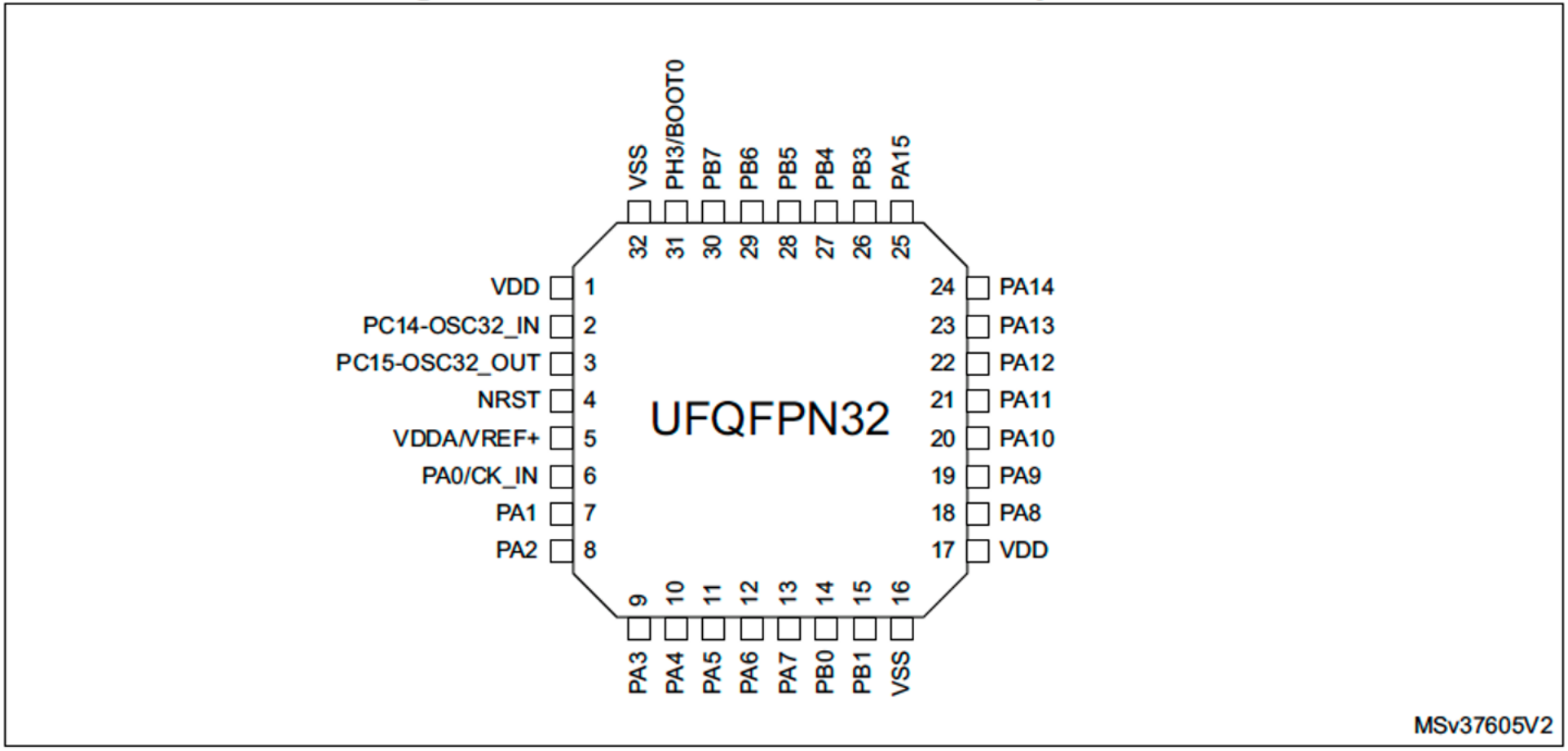
G.3 Background

- The STM **Nucleo-32** board featuring the **STM32L432KC** microcontroller (an implementation of an ARM Cortex-M4 core) is a powerful system which includes a broad range of hardware interfaces to allow for **input/output** (**I/O**) with a wide variety of peripherals.

- In this course, we use only of the microcontroller's processing and **interfacing** capabilities.

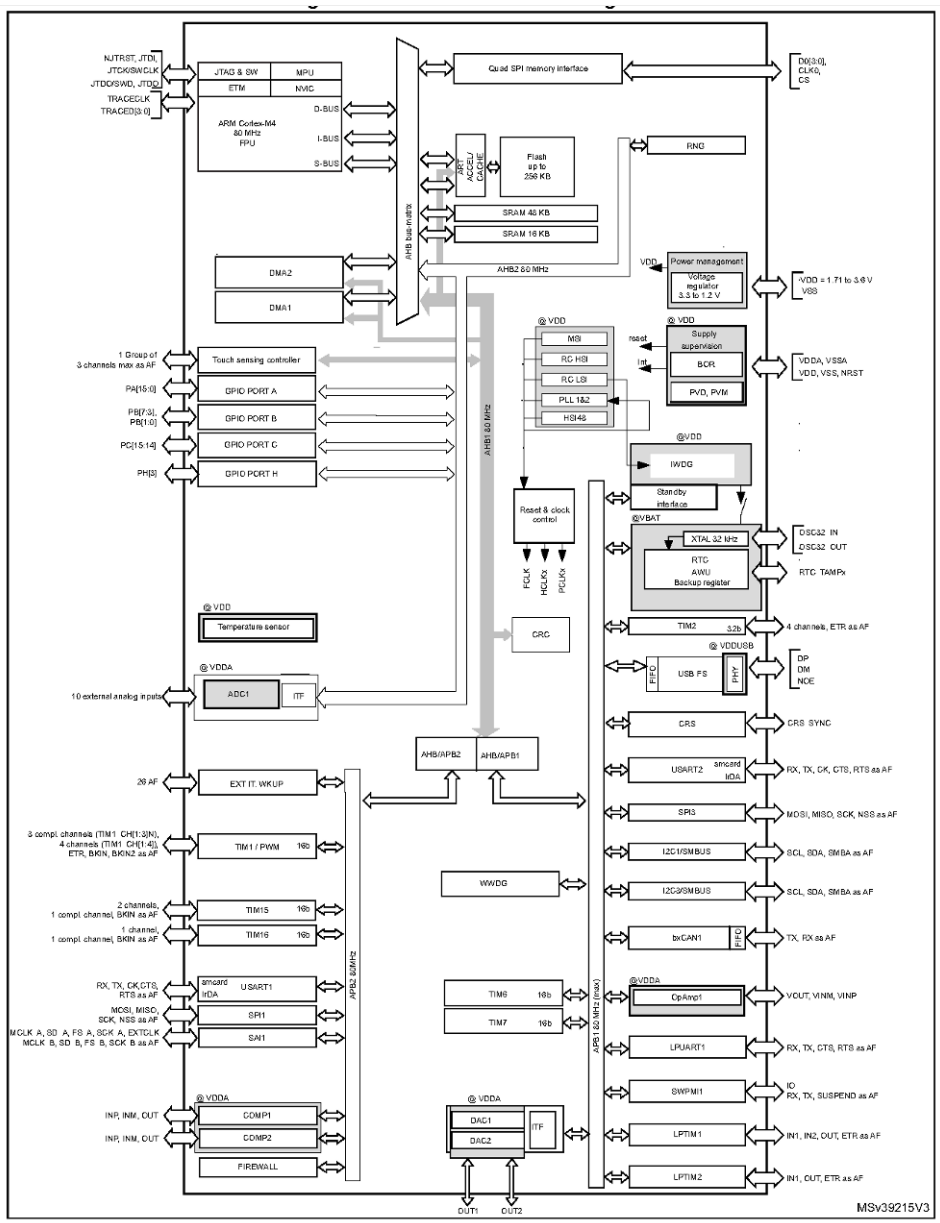
STM32L432xx microcontroller: Package and pinouts

Figure 5. STM32L432Kx UFQFPN32 pinout(1)



Ref: STM microcontroller datasheet

STM32L432xx microcontroller: Block diagram

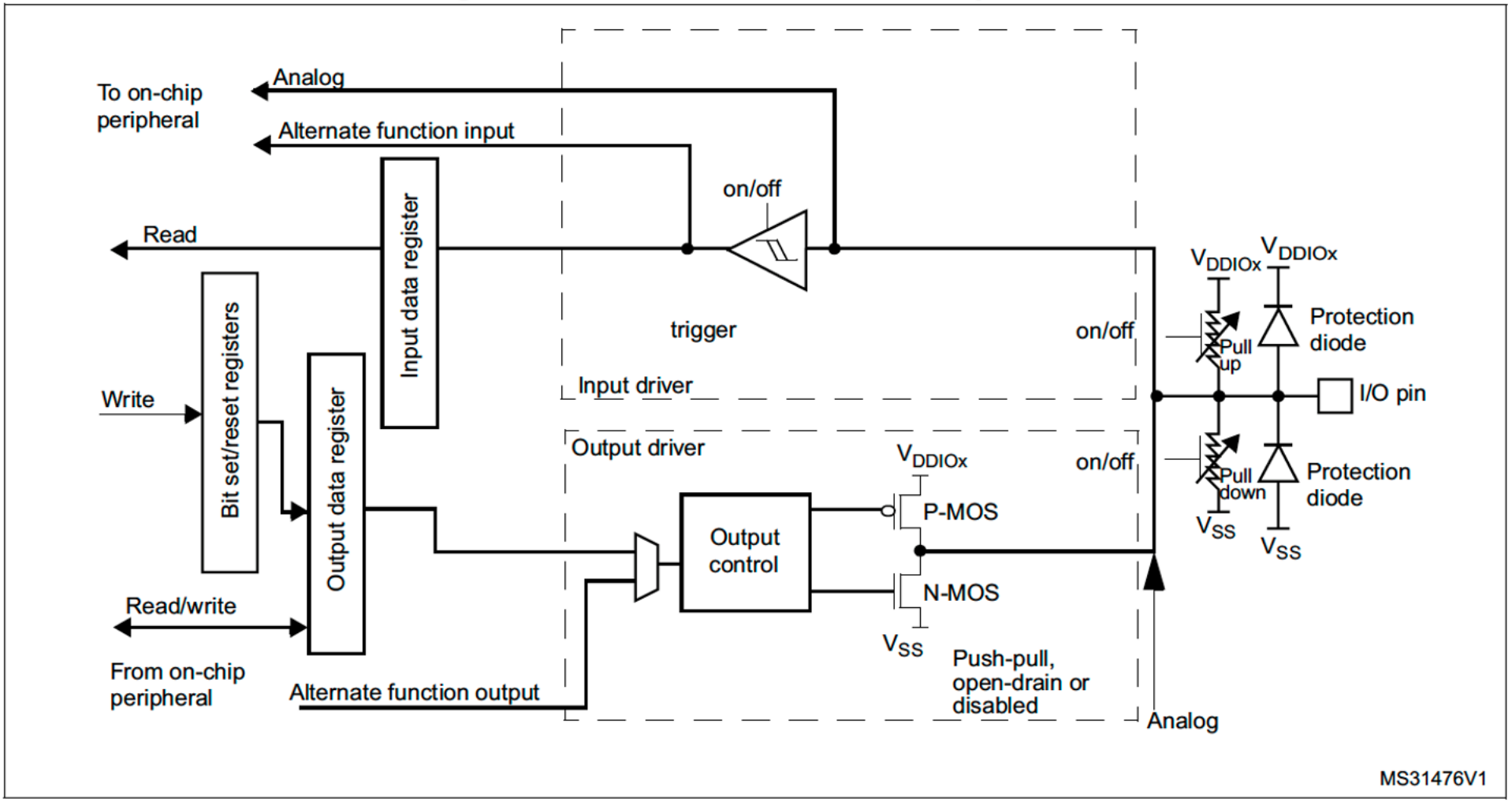


Note: AF: alternate function on I/O pins.

Ref: STM microcontroller datasheet

Many of the (physical) pins on the CPU may be used as **general-purpose input/output** (GPIO). Special sets of registers are used:

- to configure the pins for GPIO or other functions
- to configure the ports (speed, data flow direction)
- to send / receive the data itself (output / input data registers)



STM32L432xx microcontroller: So many features! 👀

Includes:

- ARM Cortex-M4 core with FPU, DSP, MPU
- 256 kB Flash memory, 64 kB SRAM
- ADC, 2× comparators, op amp, 2× DAC
- RTC, 32-bit timer, PWM timer, 4× 16-bit timer, 2× low-power timer
- 3× capacitive sensing channels
- 2× DMA
- 2× I2C
- 2× SPI
- 2× USART, low-power UART
- CAN
- USB
- SWPMI

Ref: STM microcontroller datasheet

What are all these acronyms?! 😳

- **direct memory access** (DMA) — A method by which an I/O device sends/receives data directly to/from memory, bypassing the CPU, thereby speeding up memory operations. The process is managed by the DMA controller.

```
LDR  R0,
STR  R0,
```

- **pulse-width modulation** (PWM) — a modulation technique used to encode a message into a pulsing signal
  - vary the duty cycle of a fixed-frequency square (rectangular) wave
  - Although this modulation technique can be used to encode information for transmission, one of its primary applications is for controlling power deliver; *e.g.*, to a motor.

- **joint test action group** (JTAG) — an industry standard for test and verification of printed circuit boards after manufacture

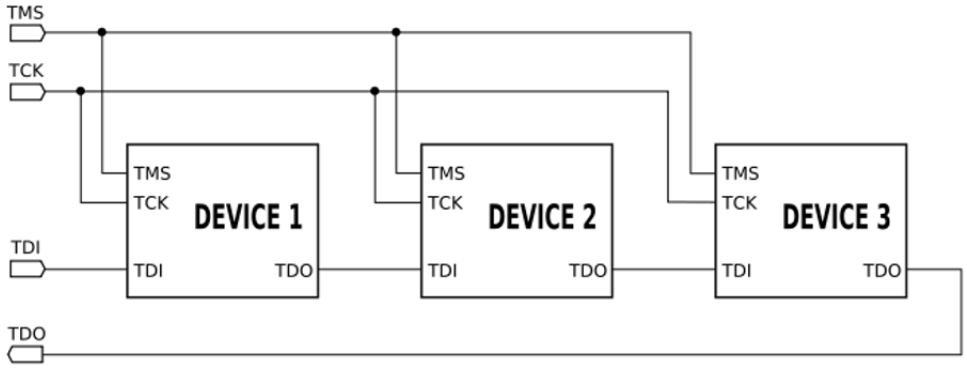
TDI = test data in

TDO = test data out

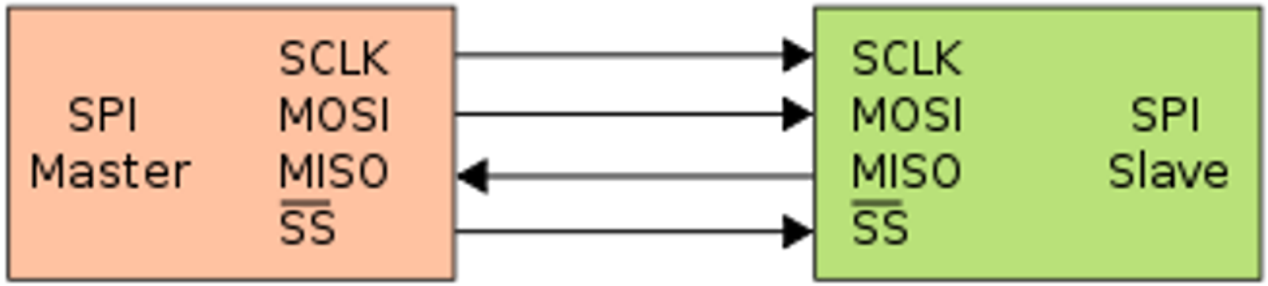
TCK = test clock

TMS = test mode select

TRST = test reset (optional)



- **serial peripheral interface** (SPI) bus — a synchronous serial communication interface specification used for short-distance communication, especially for embedded systems
  - developed by Motorola in the mid-1980s
  - SPI devices communicate in full-duplex mode using a "master-slave" architecture with a single master.



- **inter-integrated circuit** (I^2C) — a synchronous, multi-master, multi-slave, packet-switched, single-ended, serial bus
  - 1982, Philips Semiconductor (now NXP Semiconductors)
  - widely used for attaching lower-speed peripheral ICs to processors and microcontrollers in short-distance, intra-board communication

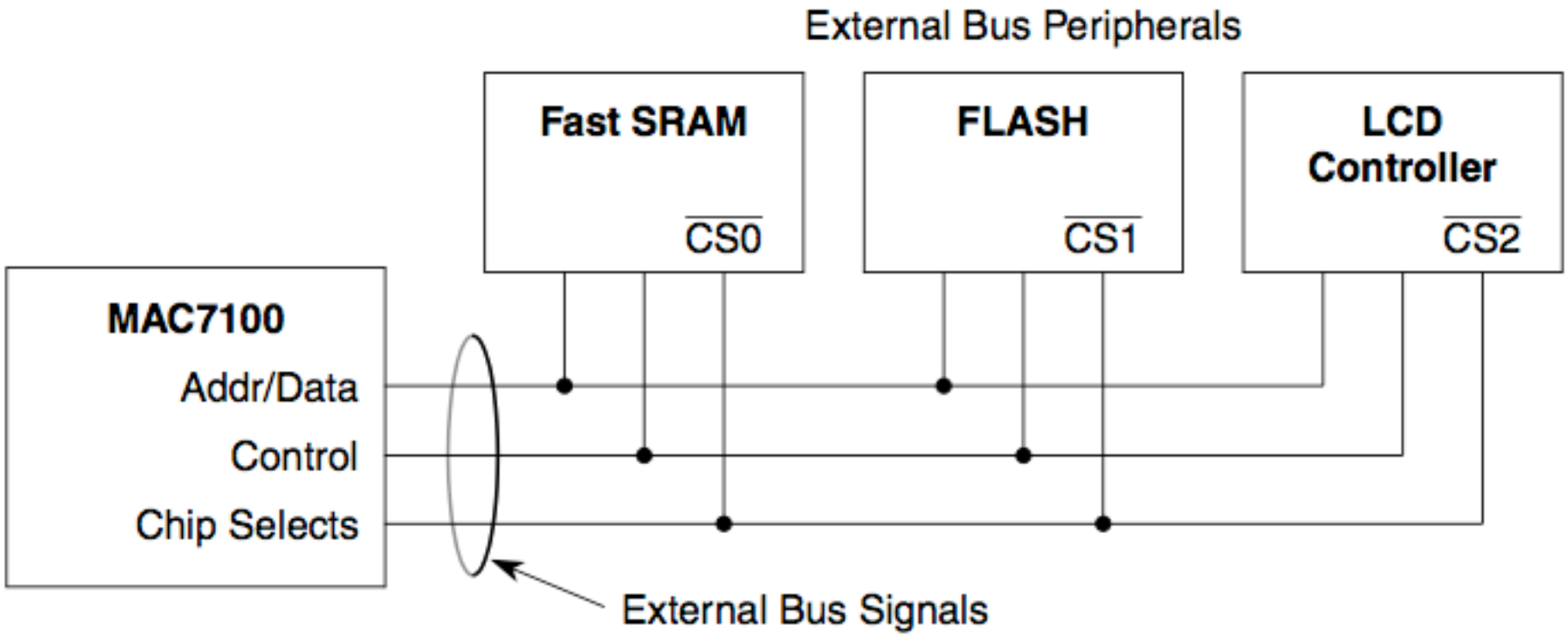
Bus signals definitions

- **active-low** — signal is "true" when logic low; indicated by signal name being complemented; *e.g.*, `AS*` or $\overline{\texttt{AS}}$
- **active-high** — signal is "true" when logic high
- **asserted** — signal is moved to its active state
- **negated** — signal is moved to its inactive state
- **tri-state signal** — a signal that can be high, low, or indeterminate (floating/disconnected)

How to get CPU and peripherals to talk to each other

- Voltage, current, and timing must be successfully reconciled between the CPU and peripherals.
- These electrical characteristics and the timing information is included in the documentation for any particular peripheral.

G.4 Memory-mapped devices



The CPU interfaces to external devices using the data and addresses busses.

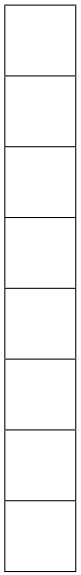
- We've thus far only considered memory (RAM).
- Other types of devices can also connect via these busses. We call them **memory-mapped devices** since they appear in the CPU's address space.
- The STM32 microcontroller has many on-chip ( ) peripherals.

G.5 Memory-to-CPU interface

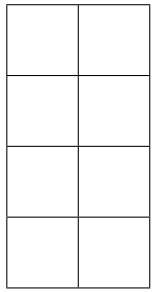
ARM Cortex-M4 core is a 32-bit processor with a 32-bit address bus and a 32-bit data bus.

Memory storage capacity is generally measured in bytes, but this capacity can be organised with different **port sizes**:

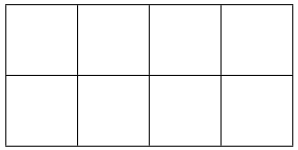
- 8-bit port — the memory can be accessed only one byte at a time



- 16-bit port — the memory can be accessed either one or two bytes at a time



- 32-bit port — the memory can be accessed one, two, or four bytes at a time

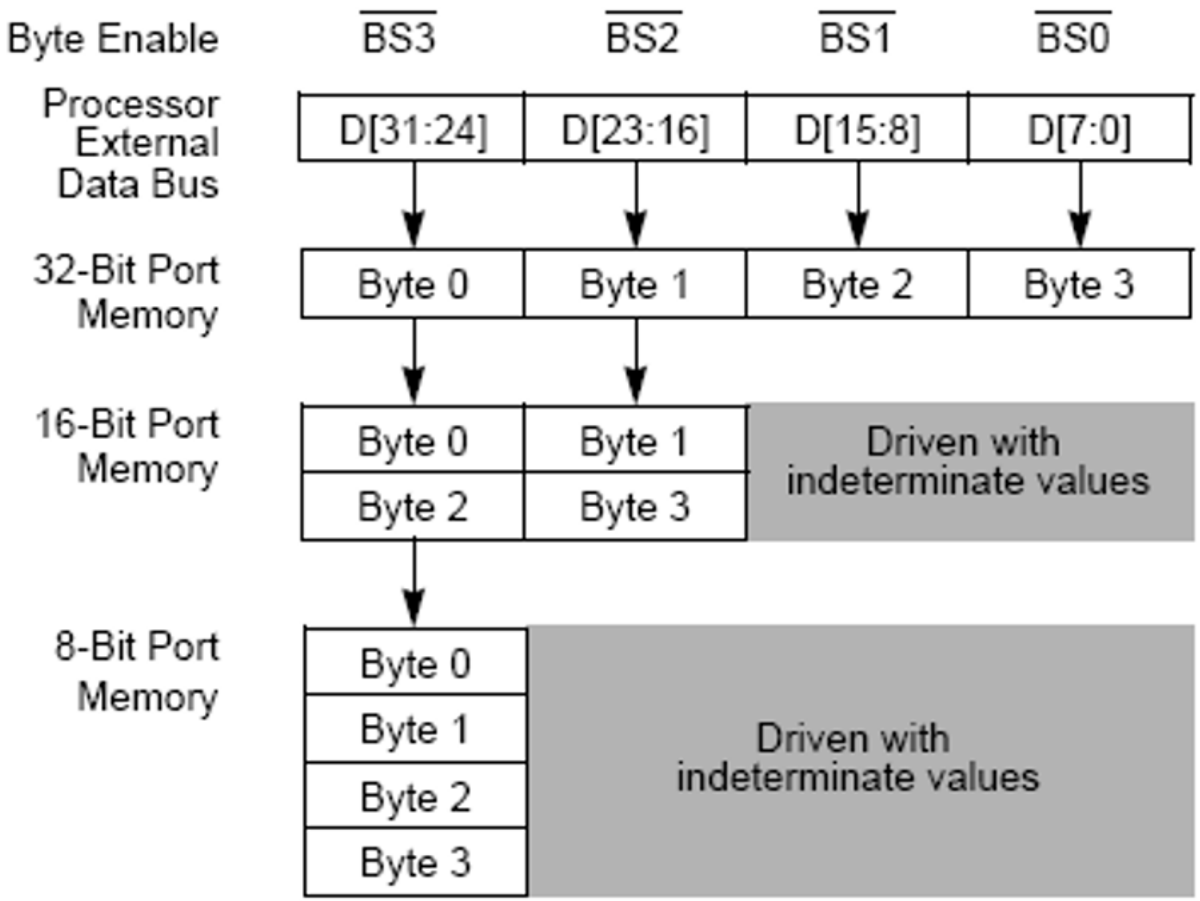


How does the memory port size map to the data bus?

- 8-bit port
  - connected to *e.g.*,
  - word-size load/store would take transfers
- 16-bit port
  - connected to *e.g.*,
  - word-size load/store would take transfers
- 32-bit port
  - connected to
  - word-size load/store would take transfer

If memory is measured in bytes and the smallest accessible unit is a byte, how is this done when the memory has a wide (*e.g.*, 32-bit) port?

- The CPU has four **byte select** (or **byte enable**) lines that specify which byte(s) of the data bus is (are) actively mapped to the memory port.
- Active-low signals $\overline{\texttt{BS3}}$ to $\overline{\texttt{BS0}}$ indicate


Byte Enable
BS3
BS2
BS1
BS0
Processor External Data Bus
D[31:24]
D[23:16]
D[15:8]
D[7:0]
32-Bit Port Memory
Byte 0
Byte 1
Byte 2
Byte 3
16-Bit Port Memory
Byte 0
Byte 1
Byte 2
Byte 3
Driven with indeterminate values
8-Bit Port Memory
Byte 0
Byte 1
Byte 2
Byte 3
Driven with indeterminate values

xfer size	port size	address		$\overline{BS3}$	$\overline{BS2}$	$\overline{BS1}$	$\overline{BS0}$
		A1	A0	D[31:24]	D[23:16]	D[15:8]	D[7:0]
byte	8-bit	0	0	0	1	1	1
		0	1	0	1	1	1
		1	0	0	1	1	1
		1	1	0	1	1	1
	16-bit	0	0	1	0	1	1
		0	1	0	1	1	1
		1	0	1	0	1	1
		1	1	0	1	1	1
	32-bit	0	0	1	1	1	0
		0	1	1	1	0	1
		1	0	1	0	1	1
		1	1	0	1	1	1
halfword	8-bit	0	0	0	1	1	1
		0	1	0	1	1	1
		1	0	0	1	1	1
		1	1	0	1	1	1
	16-bit	0	0	0	0	1	1
		1	0	0	0	1	1
	32-bit	0	0	1	1	0	0
		1	0	0	0	1	1
word	8-bit	0	0	0	1	1	1
		0	1	0	1	1	1
		1	0	0	1	1	1
		1	1	0	1	1	1
	16-bit	0	0	0	0	1	1
		1	0	0	0	1	1
	32-bit	0	0	0	0	0	0

More compact version making use of X values:

xfer size	port size	address		$\overline{BS3}$	$\overline{BS2}$	$\overline{BS1}$	$\overline{BS0}$
		A1	A0	D[31:24]	D[23:16]	D[15:8]	D[7:0]
byte	8-bit	X	X	0	1	1	1
	16-bit	X	0	1	0	1	1
		X	1	0	1	1	1
	32-bit	0	0	1	1	1	0
		0	1	1	1	0	1
		1	0	1	0	1	1
		1	1	0	1	1	1
halfword	8-bit	X	X	0	1	1	1
	16-bit	X	0	0	0	1	1
	32-bit	0	0	1	1	0	0
		1	0	0	0	1	1
word	8-bit	X	X	0	1	1	1
	16-bit	X	0	0	0	1	1
	32-bit	0	0	0	0	0	0

What these tables tell us: For a given transfer size and memory port size

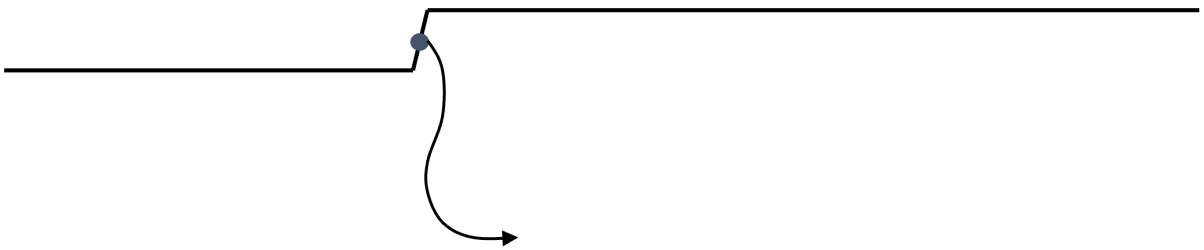
Example #1 — What will be the address and control signals to write a byte to address `0x1234` of a 32-bit port memory?

$$\begin{aligned}\text{Address} &= \texttt{0x}\,\texttt{0000}\,\texttt{1234}\\ &= \texttt{0b}\,\texttt{0000}\,\texttt{0000}\,\texttt{0000}\,\texttt{0000}\end{aligned}$$

Example #2 — Can a word be stored starting at address `0x0003`? How would the misaligned word be transferred?

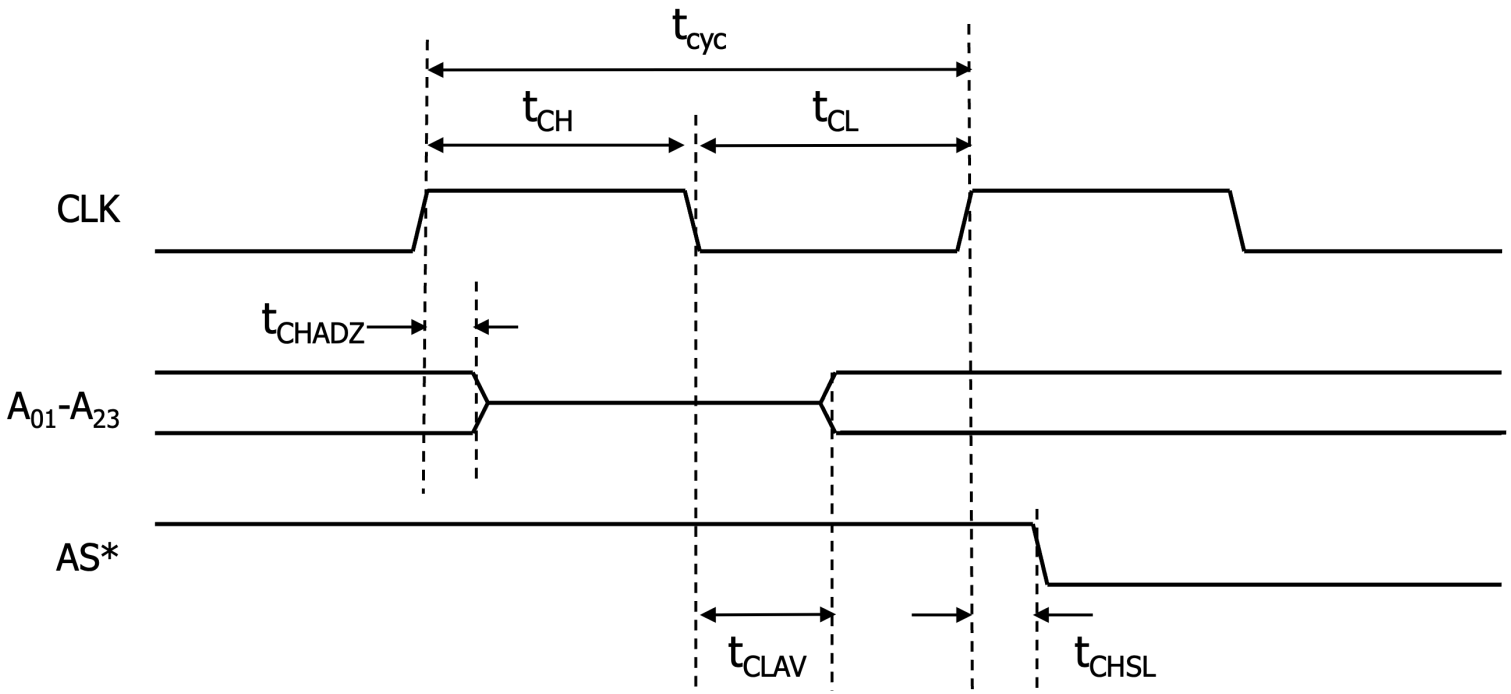
G.6 Timing diagrams

Event sequence diagrams — cause and consequence relationships between signals

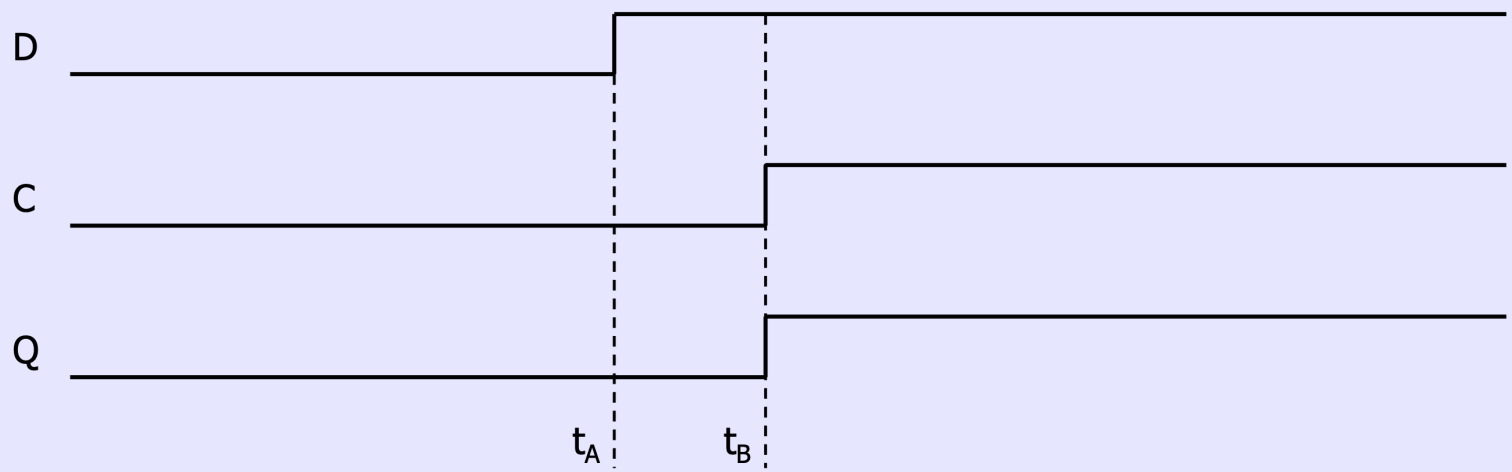


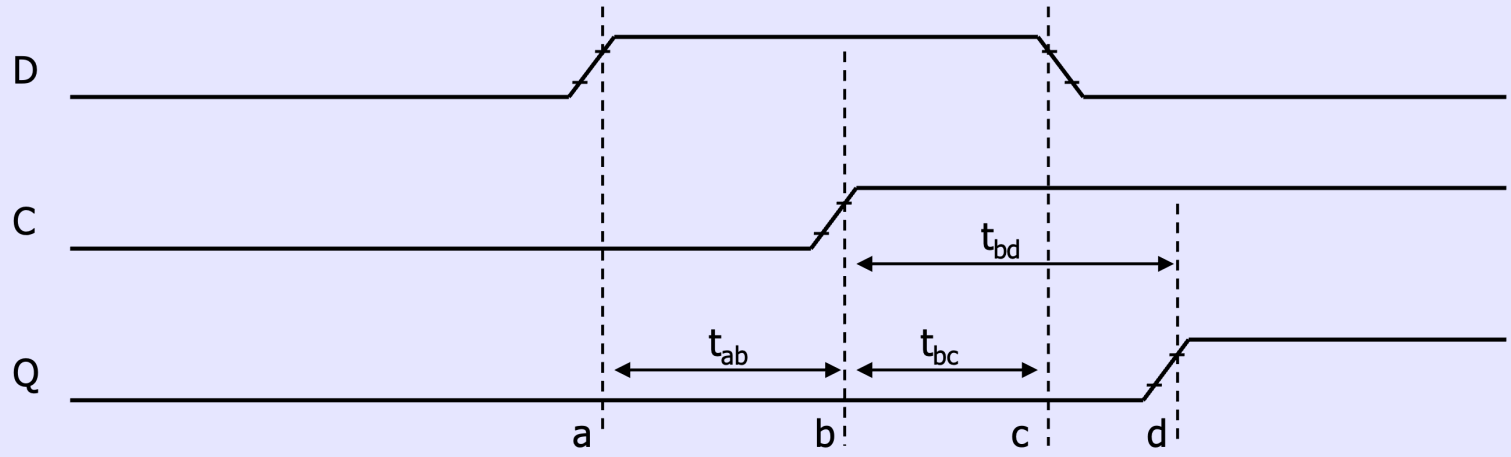
- single cause ⟶ single consequence
- multiple causes ⟶ single consequence
- single cause ⟶
- multiple causes ⟶

Data sheet timing diagram — accurately represents intervals and delays

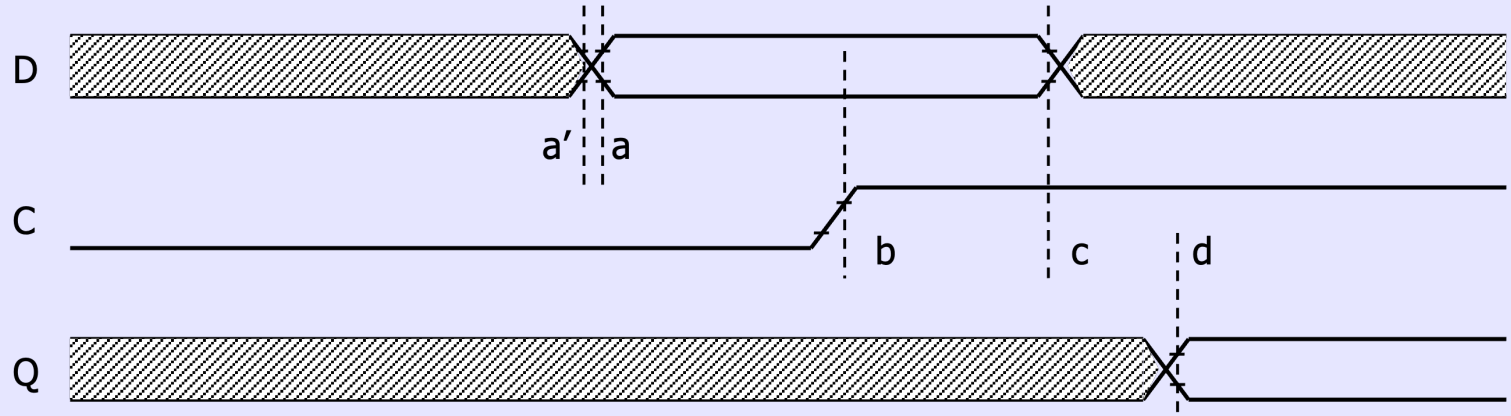


Example — Positive edge-triggered D flip-flop

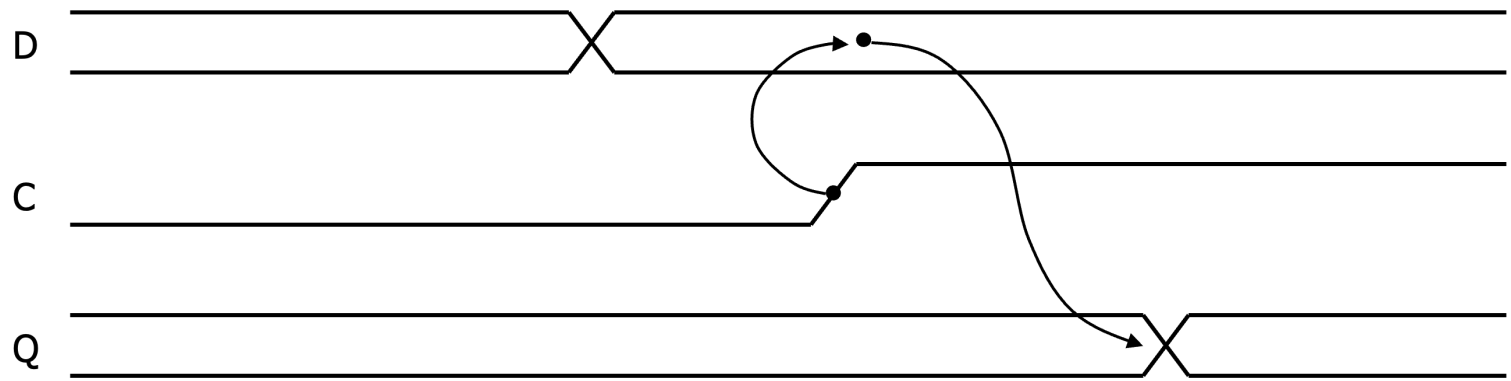




- **setup time**
  — data should be held steady for at least this long before the clock event
- **hold time**
  — data should be held steady for at least this long
- **propagation time/delay**
  — how long after clock event



General form — shaded areas indicate



Event sequence for a general combinatorial logic system:

G.7 Direct memory access (DMA)

Normally, transfers from one memory address to another memory address require the attention of the CPU.

- CPU executes instructions to transfer this data (and may require more than one instruction), which themselves must be retrieved from memory.
- The CPU isn't
- DMA is a mechanism for moving data to/from memory without needing the CPU's involvement.
- Normally, the CPU is the **bus master** for both the address bus and data bus. For DMA, the CPU this bus master role, allowing for pre-defined memory transfers; *e.g.*, disk-to-memory.

Process

1. CPU starts as the bus master.
2. A slave device requests the use of the master's buses (address, data, control).
3. Once the device has control, it can directly read from or write to system memory as needed.
4. When done, it releases the bus and the system returns to normal.

Advantages

- speed — DMA does not require CPU fetching, decoding, executing instructions
- efficiency — CPU can be relieved of the processing overhead associated with regular reading from, or writing to, I/O ports
- DMA may use otherwise unused bus cycles

Implementation is done by the **DMA controller** (DMAC), a specialised hardware device, like a simple processor.

- must emulate CPU bus cycles
- must request bus control from the CPU
- modes of operation:
  - cycle-stealing mode — bus is released between successive transfers; allows for interleaved operation with CPU
  - burst mode — DMA transfers performed on consecutive bus cycles

G.8 Address decoding

The ARM Cortex-M4 core has a memory address space of 2^{32} bytes =

- Actual physical memory and I/O devices are both mapped to and share this address space.
- Thus, multiple chips share the same data bus. However, only one chip can be active during a bus operation.
- How do we determine which chip should be active? This will depend on the the CPU needs to access. **Address decoding** is the selection (activation) of one particular chip based on the address.
- Moreover, physical memory (RAM, ROM) may be implemented using multiple ICs. The appropriate IC must be activated, supplied. This activation is controlled by chip select signals $\overline{\texttt{CSx}}$

Our 32-bit processor has an address space of 4 GB and could support a maximum of this amount of memory. But, let's say we have only 1 GB of physical memory installed, and furthermore that this is implemented as a bank of 128 MB modules — we would need .

- To be able to address individual bytes in a 128 MB memory module, we need address lines.
- Thus, split the 32-bit addresses like this:
  - The least-significant bits specify a particular address within one of the memory modules.
  - The most-significant bits would then be used to specify which of the memory modules is being accessed.

In general terms,

- The N most-significant bits used to generate $\overline{\texttt{CSx}}$ signals.
- The M least-significant bits used to select a specific memory cells within one chip.
- We can thus have blocks, each storing bytes, for a total possible storage of

  bytes

Example: Address decoding, simple case

- Consider a simple CPU that has 8-bit data bus and a 12-bit address bus.
- Memory address space is
- Assume we have four memory chips and each of them has a capacity of 1024 bytes =

For this simple computer, we can draw a memory map as:

base address = starting address for a device

In the example above, we implemented physical memory that covered the entire addressable range. This is not necessary or affordable in all cases. Two strategies:

- **full** address decoding — all address lines are used to specify a memory location and each physical memory location has a
- **partial** address decoding — only a subset of the address bus lines are needed to point to physical memory locations

Consider again the simple case, but this time choose to implement only the ROM and one 1024-byte block of RAM.

The ROM will be mapped to

Using partial address decoding, we can simplify the address decoding:

The ROM will be mapped to

In partial address decoding, some address lines are not decoded, leading to some address lines being "don't care" values.

- Multiple addresses may have the same decoding.
- Memories are mapped multiple times in memory (**echoing** or **mirroring**), as if there are multiple copies of the same memory.
- Advantages —
- Disadvantages —

Block address decoding

- Divide address space into blocks
- Fully decode each block
  - using high order bits
  - single-chip decoders available
  - *e.g.*, 2:4 decoder divides space into 4 blocks of equal size (1024 bytes for previous example)
- Optionally subdivide within block by cascading decoders with more address inputs
- Put memory device at each block start

Mixed address decoding schemes

- Full address decoding for large block; *e.g.*, decode for a large block of 1024 bytes
- Block decoding within a block; *e.g.*, use
- Partial decoding within each block; *e.g.*, I/O device data registers
- While a bit more complex, we

Address decoder design

The address decoder must assert the chip select signal whenever the address bus carries an address within the desired address range for the device. Ways to implement:

- discrete logic gates
- medium-scale integration decoder
- programmable logic devices
- *etc*.

Memory map design considerations

- The memory subspace of each device normally extends across continuous regions in the memory map.
- The regions for different devices do not necessarily have to be adjacent to each other.
- Some devices (*e.g.*, 8-bit wide I/O chips installed in systems where 32-bit words are usually accessed) may be mapped to only even/odd addresses.
- The address decoder logic must be fast; it will be used for every access into the memory.
- The mappings (and decoders) should simplify any future changes to the system.
- In small systems with large available address spaces, it is often possible to simplify the design of the address decoders by relaxing the usual requirement that each device have a unique base address.

Address decoding – Example #1

A 64 kB ROM is to be interfaced to a CPU with an 8-bit data bus and a 24-bit address bus such that the base address of the ROM is at `0x48 0000`.

- What is the full range of addresses used by the ROM?

- Design a full address decoder assuming that the ROM is on a single 64 kB chip.

Address decoding – Example #1 (cont'd)

Address decoding – Example # 2

Again, we have a CPU with an 8-bit data bus and a 24-bit address bus. The CPU is to be interfaced to a 32 kB RAM at base address `0x60 0000` and 32 kB ROM at base address `0x70 0000`.

- Design a full address decoder for this application.

Addr:	A23				A19				A15				A11				A7				A3			A0
RAM:																								
ROM:																								

Address decoder:

Address decoding – Example # 3

Again, we have a CPU with an 8-bit data bus and a 24-bit address bus. The CPU is to be interfaced to a 32 kB RAM at base address `0x60 0000` and 32 kB ROM at base address `0x70 0000`.

- Design the simplest possible address decoder.
- How many copies of the RAM and ROM will exist in the CPU's logical address space?

Addr:	A23				A19				A15				A11				A7				A3			A0
RAM:																								
ROM:																								

Address decoding – Example # 4

Again, we have a CPU with an 8-bit data bus and a 24-bit address bus. Design a simple, partial address decoding scheme for a system based on the CPU that connects to the following:

- a 32 kB EPROM **ROM1** to be placed at highest logical address relative to the other components
- a 32 kB EPROM **ROM2** at next highest address
- a 128 kB **RAM** with base address = 0
- two peripherals, **PER1** and **PER2**, each with eight internal (byte-size) registers

Addr:	A23				A19				A15				A11				A7				A3			A0
ROM1:																								
ROM2:																								

Address decoding – Example # 4.5

If the requirements are changed to:

- base addr of ROM1 and ROM2 = `0xFF 8000` and `0xFF 0000`, respectively
- base addr of RAM = `0x00 0000`
- base addr of PER1 and PER2 = `0x40 0000` and `0x60 0000`, respectively

How do we select bits for partial decoding?

Addr:	A23				A19				A15				A11				A7				A3			A0
ROM1:																								
ROM2:																								

A rule for partial address decoding: For any given pair of devices, there should be a common address line used to generate chip select signals, and whose value is 0 for one of the devices and 1 for the other device.

Address decoding – Example # 4.5 – Solution #1

Addr:	A23				A19				A15				A11				A7				A3			A0
ROM1:	1	1	1	1	1	1	1	1	1	M	M	M	M	M	M	M	M	M	M	M	M	M	M	M
ROM2:	1	1	1	1	1	1	1	1	0	M	M	M	M	M	M	M	M	M	M	M	M	M	M	M
RAM:	0	0	0	0	0	0	0	M	M	M	M	M	M	M	M	M	M	M	M	M	M	M	M	M
PER1:	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	M	M	M
PER2:	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	M	M	M

Address decoding – Example #4.5 – Solution #2

Addr:	A23				A19				A15				A11				A7				A3			A0
ROM1:	1	1	1	1	1	1	1	1	1	M	M	M	M	M	M	M	M	M	M	M	M	M	M	M
ROM2:	1	1	1	1	1	1	1	1	0	M	M	M	M	M	M	M	M	M	M	M	M	M	M	M
RAM:	0	0	0	0	0	0	0	M	M	M	M	M	M	M	M	M	M	M	M	M	M	M	M	M
PER1:	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	M	M	M
PER2:	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	M	M	M

G.9 Summary

Here, we studied computer interfacing; in particular, how the CPU communicates with other parts of the computer:

- The STM32L432KC is a microcontroller, a CPU integrated with a lot of other sub-systems
- Communication between the CPU and main memory is a crucial operation
- Devices are mapped to memory addresses, allowing facile interfacing
- Address decoding determines which external device is selected

Up next, exceptions and interrupts:

- how does the microprocessor deal with something that needs its immediate attention, disrupting normal, "linear" processing of instructions?

Course content © 2026 J. C. Sit, Electrical and Computer Engineering, University of Alberta (Canada). All rights reserved. Resale or re-distribution is prohibited.

Course content, digital or otherwise, created and/or used within the context of the course is to be used solely for personal study for students in ECE 212 LEC B1 in Winter 2026.